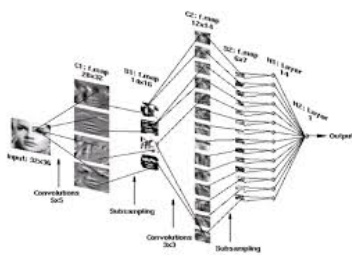


Introduction



Course Details and Topics

Adam Prügel-Bennett AICE3001/6002 Advanced Machine Learning <https://ecs-vlc.github.io/aice3001/> 1

Orientation

- This course addresses three cohorts of students:
 - ★ Third year Artificial Intelligence Students
 - ★ MSc in Machine Learning Students
 - ★ Volunteers
- It is dual coded (AICE3001/AICE6002), with subtle difference in expectations reflected in slightly different evaluation
- Notes are on Moodle and <https://ecs-vlc.github.io/aice3001/>

Adam Prügel-Bennett AICE3001/6002 Advanced Machine Learning <https://ecs-vlc.github.io/aice3001/> 3

Why Mathematics?

- Learning machines are notoriously difficult to understand
- Mathematics allows us to reason rigorously
- Posing question mathematically and seeking mathematical solutions provide allow us to build a strong intuition—we can't cheat that much
- Mathematics also allow us to describe the world precisely
- It also provides an arsenal of tools that turn out to be hugely useful

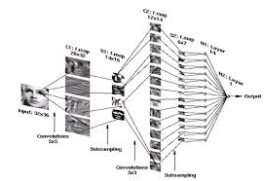
Adam Prügel-Bennett AICE3001/6002 Advanced Machine Learning <https://ecs-vlc.github.io/aice3001/> 5

Starting from Different Places

- You all start from different places
- Some of you may have a degree in mathematics
- For others it may have been your least favourite subject
- I invite strong mathematicians to become even stronger—the best way to learn is through teaching others
- For students starting from a weak base, judge yourselves by your progress, not by comparing yourself to others

Adam Prügel-Bennett AICE3001/6002 Advanced Machine Learning <https://ecs-vlc.github.io/aice3001/> 7

1. Why are you doing this course?
2. What we assume that you know



Adam Prügel-Bennett AICE3001/6002 Advanced Machine Learning <https://ecs-vlc.github.io/aice3001/> 2

Why are We Teaching You This?

- In a world where LLMs can do everything, why bother with learning mathematics?
- You need to learn to trust your own judgement
- LLMs can help us a lot, but if you don't trust yourself you are disempowered
- This course is aimed at building up trust in your understanding of how machine learning works

Adam Prügel-Bennett AICE3001/6002 Advanced Machine Learning <https://ecs-vlc.github.io/aice3001/> 4

Cracking the Code

- Mathematics is a language that takes time to master
- For some it is easy, for others it will be a struggle
- But given determination everyone can crack the code and understand what the mathematics is saying
- This comes from engaging and working with maths—this course has tutorials that should be considered compulsory—not because there are marks attached, but because without doing the work you aren't going to crack the code

Adam Prügel-Bennett AICE3001/6002 Advanced Machine Learning <https://ecs-vlc.github.io/aice3001/> 6

The Golden Rule of Mathematics

- **Understand every step**
- When you don't understand a step, stop, break the problem down, ask someone
- Mathematics uses a lot of short-cuts which speeds-up calculations but you need to be able to justify every step
- It takes bravery to admit you don't understand and insist that you will
- Asking "stupid" questions isn't stupid, it is how we all learn

Adam Prüzel-Bennett AICE3001/6002 Advanced Machine Learning <https://ecs-vlc.github.io/aice3001/> 8

- We are going to visit lots of aspects of mathematics that have proved fruitful in machine learning
- Some of this will be very concrete, while in other cases by look at a higher level of abstraction we can generalise methods across different application areas
- We will cover, calculus, vector spaces, convexity, optimisation, entropy, probability, statistics and much else besides
- The journey will pay you off, these are the foundations of most of machine learning and will give you super-powers that are hard to acquire without mathematics

What we Assume you Know

- You should all have A-level mathematics and have done some mathematics at Uni
- You should know what a function is, e.g. $f(x) = x^2 + 3$
- You should be able to do simple algebraic manipulations
- And some elementary calculus

$$f'(x) = \frac{d(x^2 + 3)}{dx} = 2x$$

Matrices

- Matrices are blocks of number that define linear mappings between vectors, e.g. $M \in \mathbb{R}^{2 \times 3}$
- $$M = \begin{pmatrix} 1.1 & -2.1 & 6.2 \\ 9.3 & 3.1 & -4.4 \end{pmatrix}$$
- An $m \times n$ matrix (i.e. $M \in \mathbb{R}^{m \times n}$) has m rows and n columns
 - An $m \times n$ matrix acting on a vector in $\mathbb{R}^n$ transforms it to $\mathbb{R}^m$ vector, e.g.

$$Mx = \begin{pmatrix} 1.1 & -2.1 & 6.2 \\ 9.3 & 3.1 & -4.4 \end{pmatrix} \begin{pmatrix} 1.2 \\ 2.3 \\ 4.7 \end{pmatrix} = \begin{pmatrix} 28.63 \\ -2.39 \end{pmatrix}$$

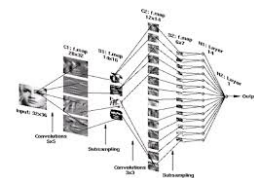
Matrix Multiplication

- The combined mapping AB can be represented by an $\ell \times n$ matrix C which we can compute using matrix multiplication (rows times columns)

$$\begin{pmatrix} \text{row 1 of } A & \text{row 2 of } A & \text{row 3 of } A & \text{row 4 of } A \end{pmatrix} \begin{pmatrix} \text{col 1 of } B \\ \text{col 2 of } B \\ \text{col 3 of } B \end{pmatrix} = \begin{pmatrix} \text{row 1 of } C & \text{row 2 of } C & \text{row 3 of } C & \text{row 4 of } C \end{pmatrix}$$

- Matrix multiplication is associative: $(AB)C = A(BC)$
- But in general not commutative $AB \neq BA$

1. Why are you doing this course?
2. What we assume that you know



Vectors

- Ordinary vectors (what mathematicians call elements of $\mathbb{R}^n$) are much used in machine learning and should be familiar
- We will use bold symbols to represent column vectors, e.g.

$$x = \begin{pmatrix} 1.2 \\ 2.3 \\ 4.7 \end{pmatrix}$$

- We denote row vectors as the transpose of column vectors $x^T = (1.2, 2.3, 4.7)$
- We care whether vectors are row or column vectors as it allows us to write equation involving vectors succinctly and unambiguously

Composition of Mappings

- Matrices, A , B , etc. represent mappings between spaces
- Provided the matrices are the correct size we can compose these mappings, e.g. if $A \in \mathbb{R}^{\ell \times m}$ and $B \in \mathbb{R}^{m \times n}$ then AB represents the effect of
 - ★ first applying B to vectors in $\mathbb{R}^n$ to obtain vectors in $\mathbb{R}^m$
 - ★ and then applying A to get vectors in $\mathbb{R}^\ell$

$$ABx = A(Bx)$$

Transpose

- We often encounter transposes of a matrix M^T where we have exchanged the rows and columns: $[M^T]_{ij} = [M]_{ji}$
- Clear $(M^T)^T = M$
- It is simple question of book keeping to show that

$$(AB)^T = B^T A^T$$
- For a symmetric matrix $M = M^T$
- Since 1×1 matrices are always symmetric

$$a^T b = (a^T b)^T = b^T a$$

$$a^T M b = b^T M^T a$$

Matrix-Vector Multiplication

- We can treat a column vector as a $n \times 1$ matrix
- Then matrix-vector multiplication can be treated the same as matrix multiplication

$$\begin{pmatrix} \square & \square & \square & \square \\ \square & \square & \square & \square \\ \square & \square & \square & \square \\ \color{red}\square & \color{red}\square & \color{red}\square & \color{red}\square \end{pmatrix} \begin{pmatrix} \color{red}\square \\ \color{red}\square \\ \color{red}\square \\ \color{red}\square \end{pmatrix} = \begin{pmatrix} \square \\ \square \\ \square \\ \color{red}\square \end{pmatrix}$$

Inner Products

- The dot or inner-product between two vectors with the same number of elements, can be written $a^T b$

$$\begin{pmatrix} \square & \square & \square & \square \end{pmatrix} \begin{pmatrix} \square \\ \square \\ \square \\ \square \end{pmatrix} = \begin{pmatrix} \square \end{pmatrix}$$

- Inner products encode angles between vectors

$$a^T b = \|a\| \|b\| \cos(\theta)$$

Outer Products

- We can also define the outer-product between any size vectors, ab^T

$$\begin{pmatrix} \square \\ \square \\ \square \\ \color{red}\square \end{pmatrix} \begin{pmatrix} \square & \square & \color{red}\square \end{pmatrix} = \begin{pmatrix} \square & \square & \square \\ \square & \square & \square \\ \square & \square & \square \\ \square & \square & \color{red}\square \end{pmatrix}$$

- Notice that we need to know whether we are dealing with row or column vectors

Norms

- Norms are non-negative numbers associated with the "size" of a vector
- The norm you should be familiar with is the **Euclidean Norm** or 2-norm

$$\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2} = \sqrt{x^T x}$$

- We will see there exist other norms (using different ways of measuring size)
- The Euclidean norm is intimately related to the inner-product, we will see other examples of this for different vector spaces

Matrices as Linear Mappings

- Matrices represent mappings between vectors

$$b = Ma$$

- The elements of b are **linear sums** of the elements of a

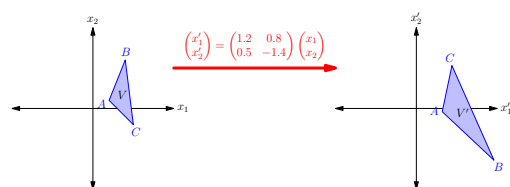
$$b_i = \sum_j M_{ij} a_j = M_{i1} a_1 + M_{i2} a_2 + \dots + M_{in} a_n$$

- Two important consequences of this linearity are for any real c

$$\begin{aligned} M(ca) &= c(Ma) \\ M(a+b) &= (Ma) + (Mb) \end{aligned}$$

Action Of Matrices on Volumes

- For a closed set of points corresponding to a volume, V in $\mathbb{R}^n$, treating each point as a vector, we can ask what happens to the volume after mapping the vector by a matrix



- For square matrices, M , the determinant is equal to $\det(M) = \pm V'/V$
- The sign tells us if there has been a reflection

Determinants

- A consequence of the fact that AB denotes the composition, apply mapping B then apply mapping A

$$\det(AB) = \det(A) \times \det(B)$$

- Determinants are only defined for square matrices, as non-square matrices project into different size spaces, where the volumes cannot be compared

- Singular matrices** projects all points onto a lower dimensional space, so have determinant 0

Multivariate Functions

- In machine learning we heavily use multivariate functions $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$
- Often our functions will be scalar value $f: \mathbb{R}^n \rightarrow \mathbb{R}$
- E.g. a learning machine for regression might be $\hat{f}(x, w)$ or we might have a loss function $L(w)$
- We say a function f is linear if

$$f(cx) = cf(x) \quad f(x+y) = f(x) + f(y)$$

- The gradient of a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ at a point $\mathbf{x}^*$ is defined as the column vector

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \frac{\partial f(\mathbf{x})}{\partial x_2} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_n} \end{pmatrix}$$

- The gradient vector is orthogonal to the contour plane and points in the direction that the function increases in fastest

Probability Distributions

- For discrete events, we can define a **probability mass function** $\mathbb{P}(X = x)$ that the random variable takes the value x
- This is a number between 0 and 1 with $\sum_{x \in \mathcal{X}} \mathbb{P}(X = x) = 1$
- For continuous events the probability of X taking a particular value x is zero
- Instead we consider the probability density $f_X(x)$ which we can define as

$$f_X(x) = \lim_{\delta x \rightarrow 0} \frac{\mathbb{P}(x < X \leq x + \delta x)}{\delta x}$$

Lesson

- I've covered material that we assume you know
- If you don't, you are going to struggle following any machine learning course, so grab a maths book and revise this
- We will revisit most of these topics to gain a better understanding, but you will be lost in the other machine learning modules if you are not familiar with everything I've covered
- There are exercises we will go through in the tutorial session where you can test your proficiency

- We will have a lot to say about probability later in this course
- To reason about events with uncertain outcomes we introduce **random variables**
- Random variables correspond to objects that we assign numbers to depending on the outcome of a random event
- E.g. we might assign 0 to the outcome of tossing a tail and 1 for a head—this is an example of a discrete random variable
- A continuous random variable might be the temperature at 12 noon tomorrow in Hyde park

Expectations

- One of the most useful quantities in dealing with random numbers is to look at the average value, aka as the **expectation**, for some function $g(X)$ of the random variable

$$\mathbb{E}[g(X)] = \begin{cases} \sum_{x \in \mathcal{X}} \mathbb{P}(X = x) g(x) \\ \int f_X(x) g(x) dx \end{cases}$$

- Expectations are linear operators

$$\begin{aligned} \mathbb{E}[cg(X)] &= c\mathbb{E}[g(x)] \\ \mathbb{E}[g(X) + h(X)] &= \mathbb{E}[g(X)] + \mathbb{E}[h(X)] \end{aligned}$$

- The expectation of a constant is itself $\mathbb{E}[c] = c$